



UNIVERSIDAD AUTÓNOMA DE CHIHUAHUA
FACULTAD DE INGENIERÍA
INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN

FUNDAMENTOS DE BASES DE DATOS

GRUPO: 6CC2

PROFESOR: José Saul De Lira Miramontes

Proyecto Final

EQUIPO: Diana Stefanya Saldaña Mendoza 376952

Erick Alfredo Ramirez Lozoya 377188

Valeria Paola Medina Ramirez 377035

Chihuahua, Chih. 26 de mayo de 2026

Manual Técnico

1. Sistema/Aplicación

a. Nombre del sistema: Besitos de Nuez

b. Descripción y delimitación del sistema: El sistema es una base de datos relacional diseñada para simular la operación de Papa's Freezeria.

El modelo permite gestionar los clientes, empleados, sucursales, el menú de productos y los pedidos, incluyendo el registro detallado de las personalizaciones de cada sundae (tamaño de vaso, mezclas, jarabes, toppings y cremas).

A través de este modelo se puede registrar, consultar y controlar el flujo completo de una orden, desde que el cliente realiza el pedido hasta su preparación con las especificaciones exactas solicitadas.

c. Objetivo general: Diseñar e implementar una base de datos relacional que permita administrar los procesos principales de operación de una heladería como el videojuego Papa 's Freezeria, facilitando el registro de clientes, empleados, pedidos y las detalladas personalizaciones de cada producto.

d. Objetivos específicos: El sistema tiene como objetivo automatizar el proceso de ventas dentro de nuestra heladería Besitos de Nuez, permitiendo registrar órdenes de manera más rápida y organizada. También busca facilitar el control de clientes, productos y tickets mediante la integración con una base de datos Oracle.

Además, la aplicación pretende mejorar la experiencia de atención al cliente al permitir la personalización de productos y la generación automática de tickets digitales.

Finalmente, se busca proporcionar una interfaz gráfica sencilla, visual e intuitiva que facilite el uso del sistema por parte de los empleados.

e. Descripción de tipos de usuarios: El sistema está diseñado para ser utilizado por empleados de la heladería, quienes interactúan directamente con el punto de venta para registrar órdenes y procesar compras de clientes.

Los empleados tienen acceso a las funciones operativas del sistema, como iniciar sesión mediante validación de ID, crear órdenes, agregar productos al carrito, registrar comentarios, procesar pagos y generar tickets. También puede consultar tickets almacenados previamente en la base de datos.

f. Entorno operativo del sistema: El sistema fue desarrollado utilizando el lenguaje de programación Python y funciona mediante una interfaz gráfica construida con la librería Tkinter.

Para la manipulación de imágenes se utiliza la librería Pillow (PIL), mientras que la conexión con la base de datos se realiza mediante la librería **oracledb**. También utiliza módulos específicos como **winsound** para la reproducción de efectos de sonido dentro de la interfaz.

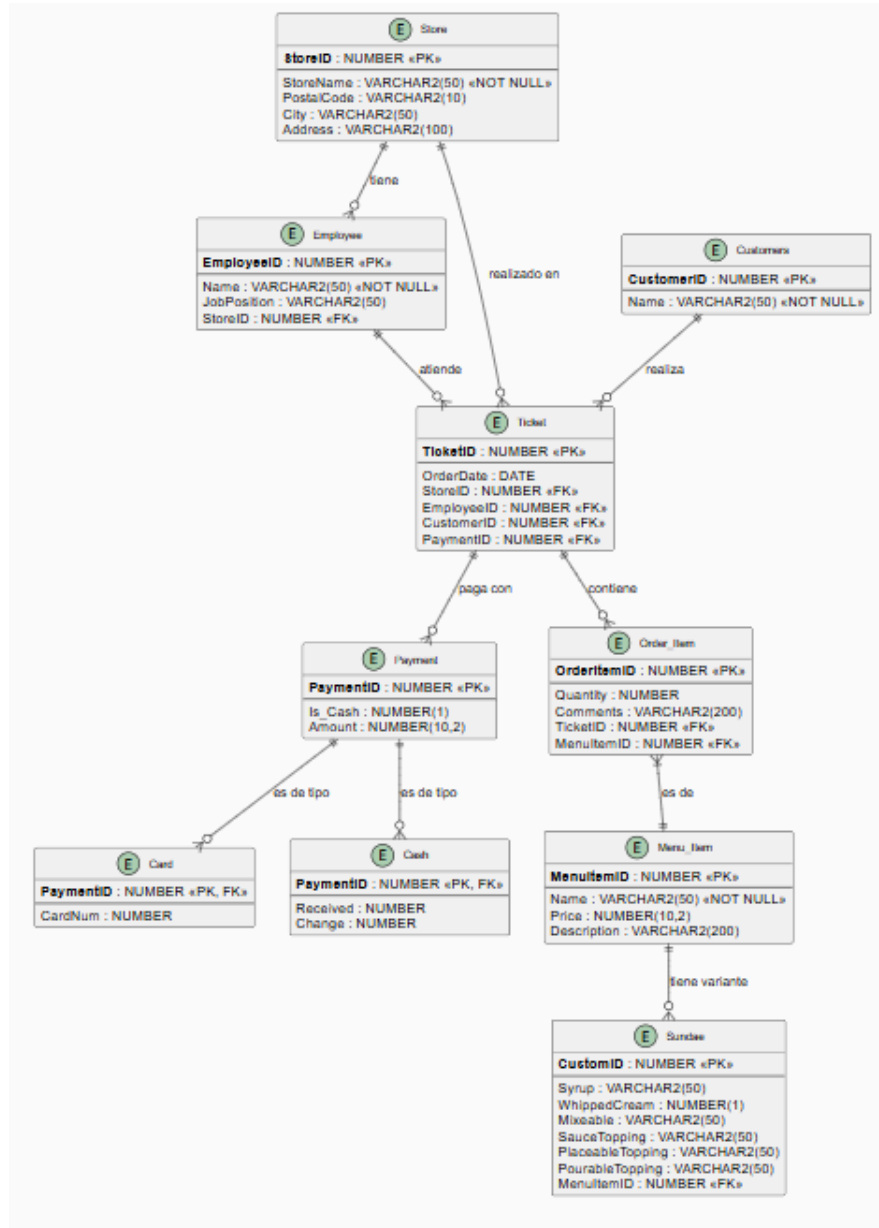
2. Especificación de requerimientos

a. Requerimientos funcionales

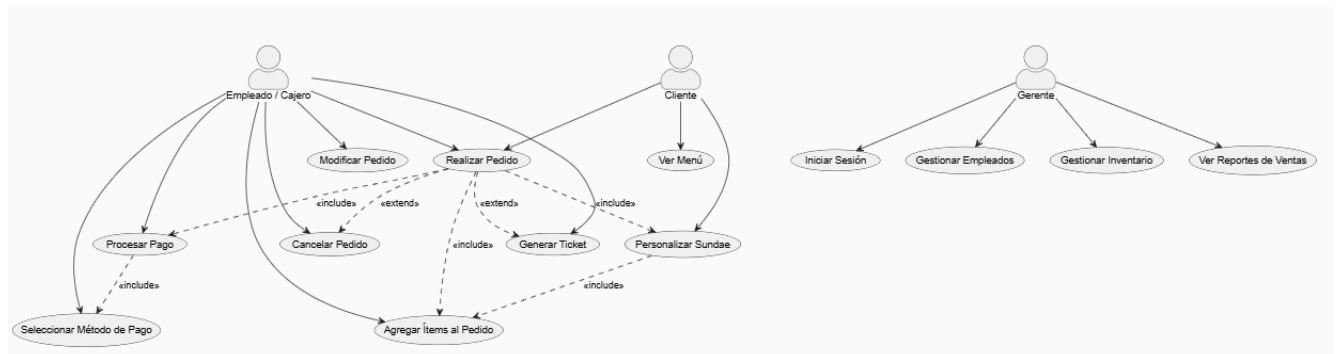
- El sistema debe permitir el acceso mediante la selección y validación de empleados registrados en la base de datos. Cada empleado deberá ingresar correctamente su ID para poder utilizar el punto de venta como método de seguridad
- La aplicación debe permitir ingresar el nombre de un nuevo cliente antes de iniciar su orden. Asimismo, debe generar automáticamente IDs únicos para clientes, tickets, pagos y productos vendidos.
- El sistema debe mostrar el menú de productos almacenado en la base de datos Oracle, incluyendo nombre, precio, ingredientes y descripción. También debe permitir visualizar imágenes asociadas a cada producto.
- La aplicación debe permitir agregar productos al carrito de compras, aumentar o disminuir cantidades y eliminar productos seleccionados. Así como permitir escribir comentarios sobre la orden. El total de la compra debe actualizarse automáticamente después de cada modificación realizada en el carrito.
- El sistema debe permitir crear productos donde el usuario podrá seleccionar distintos ingredientes para construir un sundae personalizado.
- El sistema debe manejar pagos en efectivo y tarjeta. En pagos en efectivo debe calcular automáticamente el cambio, mientras que en pagos con tarjeta debe registrar una terminación simulada de tarjeta.
- El sistema debe generar un ticket digital con toda la información de la compra, incluyendo productos, cantidades, comentarios, método de pago y total de la venta.
- La aplicación debe guardar automáticamente toda la información relacionada con la venta en la base de datos Oracle, incluyendo clientes, tickets, pagos, productos vendidos y productos personalizados.
- Finalmente, el sistema debe permitir consultar y visualizar todos los tickets almacenados previamente en la base de datos mediante una interfaz de navegación.

b. Modelado del sistema

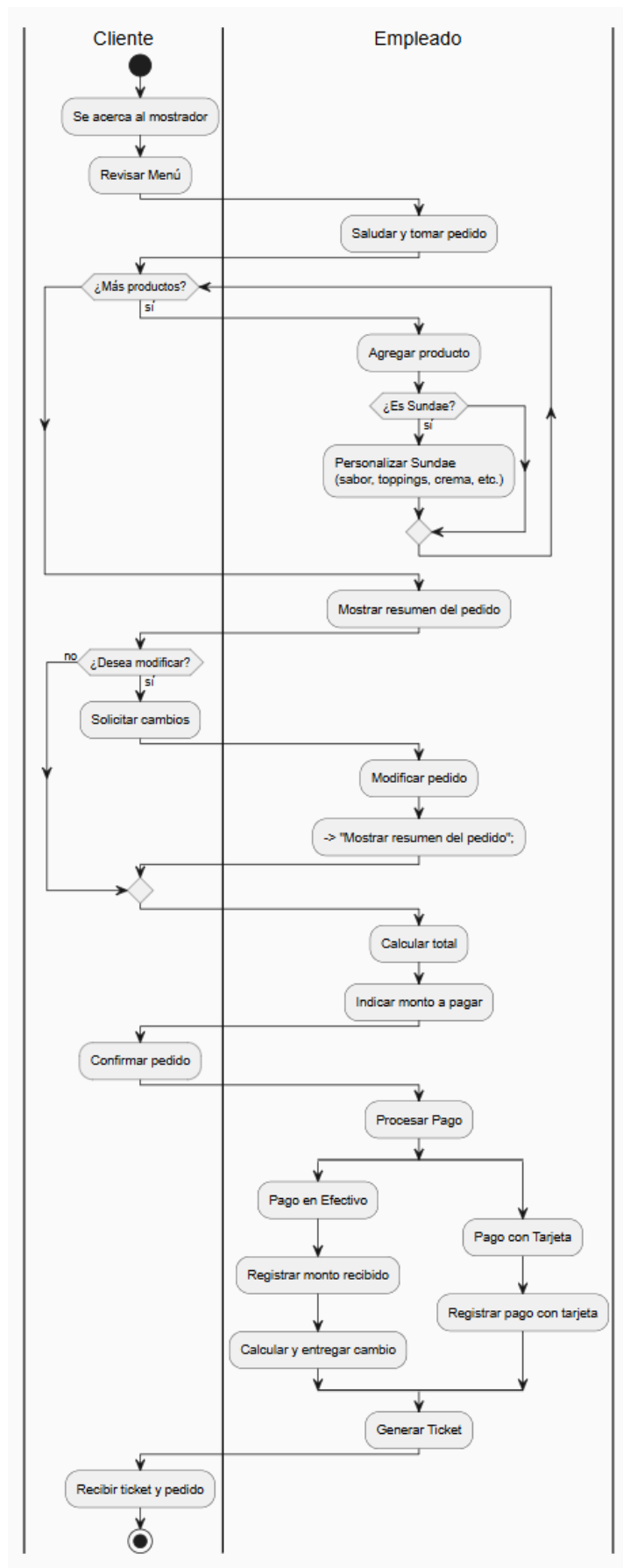
i. Diagrama de clases:



ii. Diagrama de casos de uso:



iii. Diagrama de actividades:



c. Requerimientos no funcionales

- La aplicación debe contar con una interfaz gráfica amigable, intuitiva y fácil de utilizar para cualquier empleado de la heladería.
- La aplicación debe garantizar la integridad de los datos almacenados en Oracle mediante conexiones controladas y validaciones durante las operaciones CRUD
- El sistema debe ser compatible con Python y funcionar utilizando la librería Tkinter para la interfaz gráfica, winsound para agregar los efectos de sonido y Oracle Database como gestor de base de datos.
- La aplicación debe manejar adecuadamente errores de conexión y mostrar mensajes claros al usuario cuando ocurra algún problema durante la operación.
- El sistema debe mantener la seguridad básica de acceso mediante validación de empleados y uso de consultas parametrizadas para reducir riesgos de inyección SQL.
- El sistema debe permitir mantenimiento y escalabilidad futura, facilitando la incorporación de nuevos productos y empleados sin necesidad de rediseñar completamente la aplicación.
- La aplicación debe conservar estabilidad durante la ejecución, evitando cierres inesperados o pérdida de información durante el proceso de venta.
- Finalmente, el sistema debe permitir una experiencia visual organizada mediante el uso de imágenes, botones interactivos y ventanas estructuradas para facilitar la operación diaria del punto de venta.

3. Arquitectura del Sistema/Aplicación.

a. Layer/Tier's

El proyecto está dividido en 4 capas.

Presentación: Esta capa se encarga de todo el diseño de la interfaz y todo con lo que el usuario interactúa. Todo el contenido de esta capa está incluido en el archivo **PVdiseno.py**. Se encarga de mostrar pantallas, capturar las entradas del usuario, mostrar tickets y mostrar los mensajes de error.

Lógica: Es la que se encarga de la estructura y administración del negocio.

Sus responsabilidades son:

- Validación de empleados
- Gestión del carrito
- Cálculo de totales
- Generación de tickets
- Validación de pagos
- Manejo de productos personalizados (Make your own)

Acceso de datos: Es la encargada de comunicarse con la base de datos Oracle. Se encuentra en HeladeriaBD_CRUD.py

Realiza:

- Conexión a la BD
- Consultas SQL
- Inserciones
- Lectura de datos
- Recuperación de tickets y productos

Base de datos: Es la que se encarga de cargar los datos a la aplicación, ahí se encuentran todas las tablas y por lo tanto se incluye los datos de los empleados, de los productos, de los tickets, de los métodos de pago, etc.

Su función es almacenar la información de las ventas y productos junto con mantener la integridad de los datos.

b. Frontback/Backend.

Para Backend:

Para el backend utilizaremos python para la parte lógica del programa, junto con SQL Developer de Oracle para manejar la base de datos y sus elementos.

Decidimos usar python ya que ofrece una sintaxis sencilla, flexible y eficiente para la creación de aplicaciones orientadas a la gestión de datos. Python nos permitirá implementar toda la lógica del sistema, incluyendo el manejo de pedidos, validaciones de información, registro de clientes, administración de empleados y procesamiento de las personalizaciones de los productos.

Además, facilita la conexión con bases de datos relacionales mediante librerías especializadas, las cuales permiten establecer comunicación directa con Oracle Database. Gracias a esto, el sistema podrá realizar operaciones CRUD y consultas complejas.

Por otra parte, Oracle SQL Developer será utilizado como herramienta principal para diseñar y administrar la base de datos relacional. En esta plataforma se crearán las tablas, relaciones, claves primarias y foráneas, restricciones de integridad y consultas SQL necesarias para el funcionamiento del sistema.

Para Frontend:

En el Frontend haremos uso de python para la interfaz con ayuda de librerías como Tkinter para crear ventanas, botones, darle color, entre otros recursos. Tkinter facilitará el diseño de una interfaz amigable e intuitiva, permitiendo que los usuarios puedan navegar fácilmente entre las distintas funciones del sistema.

A través de esta interfaz será posible registrar clientes, visualizar el menú de productos, crear pedidos y consultar información almacenada en la base de datos. La interfaz gráfica va a estar organizada mediante diferentes ventanas y módulos para mejorar la experiencia del usuario.

c. Estructura modular del sistema

Módulo de Interfaz Gráfica

Archivo: PVfuncionamiento.py

Función: Controla todas las pantallas y la interacción del usuario. Fue desarrollado con Tkinter y contiene la clase principal Punto Venta Heladeria, responsable de administrar las distintas pantallas y el flujo general de la aplicación.

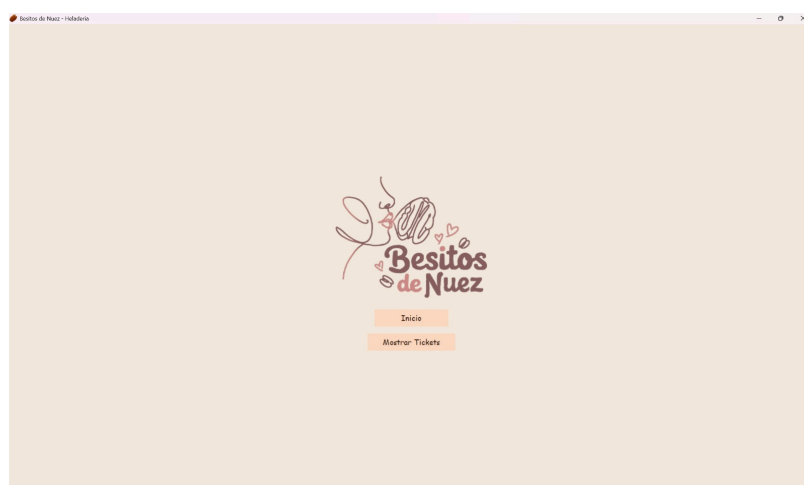
Módulo de Diseño Visual

Archivo: PVdiseno.py

Contiene:

- Colores
- Fuentes
- Estilos
- Botones personalizados
- Imágenes
- Componentes visuales

Ejemplo visual:



Módulo de Acceso a Datos

Archivo:

HeladeriaBD_CRUD.py

Contiene funciones CRUD para Oracle Database.

Funciones:

- conectar()
- crear_customer()
- crear_payment()
- crear_ticket()
- crear_order_item()
- leer_menu_items()
- leer_menu_item_por_id()
- leer_tickets()
- leer_ticket_por_id()
- leer_payments()
- leer_payment_por_id()
- obtener_id_gerente()
- leer_employees()
- leer_employee_por_id()
- leer_stores()
- leer_store_por_id()

Módulo de Gestión de Ventas

Métodos principales

- crear_sundae()
- leer_sundaes()
- leer_sundae_por_id()
- actualizar_sundae()
- eliminar_sundae()

Módulo de Productos Personalizados

Método principal

- mostrar_custom()

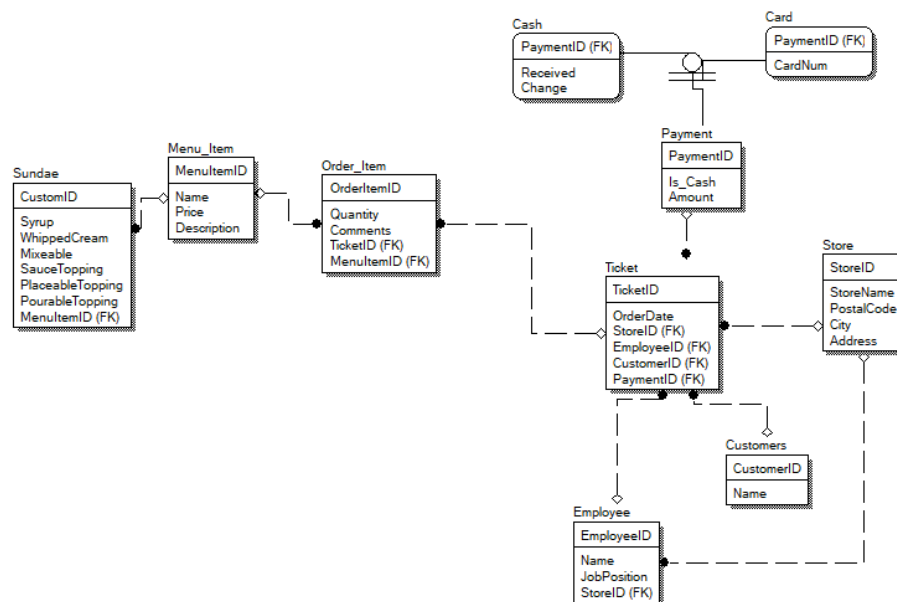
Módulo de Tickets

Métodos principales

- leer_tickets()
- leer_ticket_por_id()
- leer_order_items()
- leer_order_item_por_id()
- leer_payments()
- leer_payment_por_id()

4. Base de datos

a. Diagrama Conceptual (Entidad/Relación)



b. Esquema Lógico de la Base de Datos

i. Especificación Tablas (Normalizadas hasta BCNF)

Tabla	Atributos (Columnas)	Tipo de Dato	Clave Primaria	Forma Normal
Payment	PaymentID Is_Cash Amount	NUMBER NUMBER(1) NUMBER(10,2)	PaymentID	BCNF
Card	PaymentID CardNum	NUMBER NUMBER	PaymentID	BCNF
Cash	PaymentID Received Change	NUMBER NUMBER	PaymentID	BCNF

		NUMBER		
Store	StoreID StoreName PostalCode City Address	NUMBER VARCHAR2(50) VARCHAR2(10) VARCHAR2(50) VARCHAR2(100)	StoreID	BCNF
Employee	EmployeeID Name JobPosition StoreID	NUMBER VARCHAR2(50) VARCHAR2(50) NUMBER	EmployeeID	BCNF
Customers	CustomerID Name	NUMBER VARCHAR2(50)	CustomerID	BCNF
Ticket	TicketID OrderDate StoreID EmployeeID CustomerID PaymentID	NUMBER DATE NUMBER NUMBER NUMBER NUMBER	TicketID	BCNF
Menu_Item	MenuItemID Name Price Description	NUMBER VARCHAR2(50) NUMBER(10,2) VARCHAR2(200)	MenuItemID	BCNF
Order_Item	OrderItemID Quantity Comments TicketID MenuItemID	NUMBER NUMBER VARCHAR2(200) NUMBER NUMBER	OrderItemID	BCNF
Sundae	CustomID Syrup WhippedCream Mixeable SauceTopping PlaceableTopping PourableTopping MenuItemID	NUMBER VARCHAR2(50) NUMBER(1) VARCHAR2(50) VARCHAR2(50) VARCHAR2(50) VARCHAR2(50) NUMBER	CustomID	BCNF

ii. Integridad de Datos (Constraints)

Tabla Hija	Columna FK	Referencia a	Acción al Eliminar
Card	PaymentID	Payment(PaymentID)	ON DELETE CASCADE
Cash	PaymentID	Payment(PaymentID)	ON DELETE CASCADE
Employee	StoreID	Store(StoreID)	ON DELETE SET NULL
Ticket	StoreID	Store(StoreID)	ON DELETE SET NULL
Ticket	EmployeeID	Employee(EmployeeID)	ON DELETE SET NULL
Ticket	CustomerID	Customers(CustomerID)	ON DELETE SET NULL
Ticket	PaymentID	Payment(PaymentID)	ON DELETE SET NULL
Order_Item	TicketID	Ticket(TicketID)	ON DELETE SET NULL
Order_Item	MenuItemID	Menu_Item(MenuItemID)	ON DELETE SET NULL
Sundae	MenuItemID	Menu_Item(MenuItemID)	ON DELETE SET NULL

5. Lógica/Reglas del Negocio

El sistema fue diseñado para simular el funcionamiento de un punto de venta real dentro de una heladería. La lógica principal consiste en controlar el flujo completo de una venta, desde la selección del empleado hasta la generación del ticket y el almacenamiento de la información en la base de datos Oracle.

El proceso inicia con un menú que incluye dos opciones. Puedes **iniciar** la selección y validación de un empleado registrado en la base de datos donde el sistema solicita el ID del empleado para verificar su identidad antes de permitir el acceso al punto de venta. Una vez validado, el empleado puede crear una nueva orden ingresando el nombre del cliente. O también puedes **mostrar** todos los tickets almacenados en la base de datos.

Cada orden genera automáticamente identificadores únicos para el cliente, el ticket y el pago. Posteriormente, el usuario puede agregar productos al carrito desde el menú cargado dinámicamente desde la base de datos. El sistema permite modificar cantidades y eliminar productos, agregar comentarios, escoger el método de pago, iniciar una nueva orden o cancelar el pedido.

Una de las funciones principales del sistema es la creación de productos personalizados mediante la opción "Make your own". Esta funcionalidad permite seleccionar de entre distintos ingredientes para construir un sundae personalizado.

El sistema maneja dos métodos de pago: efectivo y tarjeta. Cuando el pago es en efectivo, el sistema verifica que el monto ingresado sea suficiente y calcula automáticamente el cambio. En pagos con tarjeta, se genera un número parcial simulado para representar la terminación de la tarjeta utilizada.

Finalmente, al concluir la compra, el sistema guarda toda la información relacionada con la venta en las tablas correspondientes de Oracle, incluyendo cliente, ticket, productos comprados, pago y personalizaciones realizadas. Después se genera un ticket visual que resume toda la operación realizada.

6. Descripción Interfaz de la aplicación

La interfaz gráfica fue desarrollada por la librería Tkinter de Python. El diseño utiliza colores suaves, botones personalizados e imágenes para representar los productos del menú.

La aplicación inicia con una pantalla principal sencilla que muestra el nombre del sistema y las opciones principales para iniciar una venta o consultar tickets previamente almacenados. A partir de ahí, el usuario puede seleccionar un empleado desde una lista cargada automáticamente desde la base de datos.

Después de validar al empleado, se muestra una ventana para registrar el nombre del cliente y comenzar una nueva orden. Posteriormente se abre la interfaz principal del punto de venta, la cual se divide en diferentes secciones para facilitar la operación.

En la parte izquierda se encuentra la información general de la venta, como el empleado activo, el nombre del cliente, el número de ticket, el carrito de productos y el total acumulado. También se incluyen botones para aumentar o disminuir cantidades, eliminar productos y realizar el pago.

En la parte derecha se encuentra el menú de productos, donde cada artículo puede mostrarse mediante imágenes obtenidas desde una carpeta local. Al seleccionar un producto, el sistema muestra su descripción e ingredientes en una ventana inferior. Esto permite que el usuario consulte los detalles del producto..

La opción "Make your own" abre una ventana independiente donde el usuario puede personalizar un sundae seleccionando distintos ingredientes mediante listas desplegables. Esta interfaz facilita la creación de productos personalizados de manera visual y organizada.

Una vez completada la compra, el sistema muestra una pantalla de ticket con toda la información de la venta, incluyendo productos, cantidades, comentarios, ingredientes personalizados, método de pago y total, en caso de haberlo pagado con efectivo también muestra el cambio a dar. Además, existe una sección para visualizar tickets guardados previamente en la base de datos y navegar entre ellos.

7. Descripción de reglas de seguridad (acceso/operación)

El acceso al sistema está restringido únicamente a empleados registrados. Antes de ingresar al punto de venta, el usuario debe validar correctamente el ID correspondiente al empleado seleccionado. Esto evita que usuarios no autorizados puedan utilizar la aplicación.

Durante las operaciones del sistema se realizan múltiples validaciones para evitar errores o inconsistencias. Por ejemplo, no es posible generar una venta sin productos en el carrito ni continuar una orden sin registrar un cliente. De igual manera, en pagos en efectivo el sistema verifica que el monto proporcionado sea suficiente para cubrir el total de la compra.

Otra medida importante consiste en la generación automática de identificadores para tickets, pagos y órdenes, evitando duplicados manuales o errores de captura.

8. Conclusión

En conclusión, con el proyecto Besitos de Nuez pudimos aplicar todo lo aprendido en la materia de Fundamentos de Bases de Datos. Logramos crear una base de datos funcional en Oracle y una aplicación en Python con interfaz gráfica que simula el funcionamiento de una heladería.

Durante el desarrollo pudimos implementar el registro de pedidos, la personalización de sundaes, el manejo de pagos y la generación de tickets, todo conectado correctamente con la base de datos. Aunque todavía hay algunas partes que se pueden mejorar, el sistema cumple con la mayoría de los requerimientos que nos propusimos.

Este proyecto nos ayudó a entender mejor cómo se relacionan las tablas, la importancia de la normalización y cómo conectar Python con Oracle. Además de la importancia de diseñar correctamente la base de datos desde el inicio para en etapas futuras no tener que volver al inicio para realizar correcciones.

Estamos satisfechos con el resultado obtenido y contentos de haber terminado nuestro proyecto final.

9. Anexos

Images

- AppLogo.png
- Capricho de la Abuela.png
- Choco-Fresa.png
- Choco-Rolo.png
- Cocazo de Crema.png
- Desvelado de Olla.png
- Don Churro.png
- Dulces Besos.png
- Ensueño Tropical.png
- Logo.png
- Luna de Octubre.png
- Make your own.png
- MazaPlatano.png
- Mora-Mor.png
- Pasion Roja.png
- Pica-Mango.png
- Rosa Mexicano.png
- Taro con Moras.png
- Ticket.png

Sounds

- SonidoBoton.wav
- STicket.wav

HeladeriaBD_CRUD.py

PVdiseno.py

PVfuncionamiento.py

10. Bibliografía/Referencias

Erwin, Inc. (s. f.). erwin Data Modeler Documentation.

<https://bookshelf.erwin.com/>

Oracle. (s. f.). Oracle Database Documentation. Oracle Help Center.

<https://docs.oracle.com/en/database/>

Oracle. (s. f.). python-oracledb Documentation. Oracle.

<https://python-oracledb.readthedocs.io/>

Python Software Foundation. (s. f.). tkinter - Python interface to Tcl/Tk.

<https://docs.python.org/3/library/tkinter.html>

Python Software Foundation. (s. f.). datetime - Basic date and time types.

<https://docs.python.org/3/library/datetime.html>

Lucidchart. (s. f.). Database Modeling and Entity Relationship Diagrams.

<https://www.lucidchart.com/pages/database-diagram/database-models>